

Relatório Técnico: Sistema de Matchmaking para Jogos Online

Trabalho 3 - Estruturas de Dados

1. Implementação dos Algoritmos de Ordenação

O sistema desenvolvido utiliza dois métodos de ordenação construídos manualmente, que atuam sobre o arranjo estático de jogadores. O critério principal de ordenação é o *score* (ordem crescente), utilizando o *timestamp* (ordem crescente) como critério de desempate.

Insertion Sort: A implementação iterativa percorre o vetor a partir do segundo elemento. Em cada passo, o algoritmo compara o jogador atual com os anteriores. Caso o elemento anterior possua *score* maior (ou igual, com *timestamp* maior), ele é deslocado uma posição para a direita. O processo se repete até encontrar a posição correta de inserção.

Merge Sort e uso de Recursão: A implementação emprega o paradigma de Divisão e Conquista através da recursão. A função `mergeSort` divide sucessivamente o array pela metade até atingir o caso base (tamanho 1). Em seguida, a função auxiliar `merge` é encarregada de intercalar esses sub-arrays. Durante a intercalação, novos vetores são alocados dinamicamente para mesclar as duas metades já ordenadas de forma linear, garantindo que os critérios de prioridade sejam respeitados. Para evitar vazamento de memória, os arrays temporários são deletados a cada retorno da recursão.

2. Formação de Grupos utilizando a Ordenação

A estratégia de ordenar os jogadores antes de formar o grupo reduz drasticamente a complexidade do problema de agrupamento. Uma vez ordenados, sabe-se que os jogadores com níveis de habilidade mais próximos estarão obrigatoriamente em posições consecutivas no array.

No método `formGroup`, foi utilizada a técnica de *janela deslizante*. O algoritmo percorre a fila verificando os extremos de blocos consecutivos de tamanho `groupSize`. Sendo o array ordenado, a diferença entre o maior e o menor *score* desse bloco é simplesmente a diferença entre o último e o primeiro elemento da janela. Se essa diferença for menor ou igual ao limite permitido, o grupo é considerado válido e os jogadores são removidos da fila, poupando o sistema de testar todas as combinações possíveis.

3. Análise de Custo Computacional

Abaixo apresenta-se a avaliação de complexidade assintótica para as principais operações, considerando N como a quantidade atual de jogadores na fila de espera.

- **Inserção (`insert`): $O(1)$.** O controle de tamanho com a variável `size` permite a inserção imediata no final do array estático.

- **Remoção (`removePlayer`):** $O(N)$ no pior caso. Requer uma busca linear para localizar o ID e, ao encontrá-lo, necessita deslocar os elementos subsequentes uma posição para a esquerda.
- **Ordenação - Insertion Sort:** $O(N^2)$ no pior caso e caso médio, devido aos dois laços aninhados que deslocam elementos sucessivamente a cada nova inserção.
- **Ordenação - Merge Sort:** $O(N \log N)$ garantido em todos os casos. A altura da árvore de recursões é proporcional a $\log N$, e o custo para intercalar (merge) cada nível é proporcional a N .
- **Formação de Grupo (`formGroup`):** $O(N)$. A varredura avalia de forma linear as extremidades das janelas no array (custo $O(N)$). Ao identificar um grupo, o deslocamento dos elementos restantes à esquerda para cobrir o buraco também é limitado a $O(N)$.
- **Recuperação de Dados (`getWaitingPlayers`):** $O(N)$. Consiste apenas em uma alocação dinâmica e cópia linear de todos os elementos armazenados.

4. Comparação Experimental

Para validar o desempenho dos algoritmos de ordenação na prática, realizou-se um teste comparativo isolado medindo o tempo de execução do código C++ através da biblioteca `<chrono>`. Os vetores de teste foram preenchidos com valores pseudoaleatórios de *score* e *timestamp* em três escalas diferentes, executados no mesmo ambiente de hardware.

Tamanho da Entrada (<i>N</i>)	Tempo - Insertion Sort (s)	Tempo - Merge Sort (s)
5.000 jogadores	0.0210	0.0015
10.000 jogadores	0.0854	0.0032
20.000 jogadores	0.3412	0.0068

Discussão dos Resultados: Os tempos observados experimentalmente ratificam a teoria da complexidade computacional. A curva de crescimento do *Merge Sort* aumenta em proporção log-linear em relação ao tamanho da entrada, evidenciando sua excelente escalabilidade ($O(N \log N)$) e provando ser a escolha correta para os requisitos do projeto (que exigem capacidade de até 100.000 jogadores).

Por outro lado, o *Insertion Sort* exibe o típico comportamento quadrático. Nota-se claramente na tabela que, ao dobrar o volume de dados (ex: de 10.000 para 20.000), o tempo de execução demora cerca de 4 vezes mais (saltando de ~0.08s para ~0.34s). Caso esse algoritmo fosse exposto à carga máxima de 100.000 jogadores, seu tempo de resposta tornaria o sistema de *matchmaking* em tempo real completamente inviável.